Experiment No. 3

Name: Devang Deokule
Roll Number: A-58
TU ID: TU4S2425009

AIM: Implementation of any Classifier using Open source tool WEKA/Rapid Miner.
a) Implementation of Decision Tree Classifier using WEKA.
b) Implementation of Random Forest Classifier using WEKA

A) Implementation of Decision Tree Classifier using WEKA.

Objectives:

After completing this experiment you will be able to:

1. List the differences among the learning types: supervised and unsupervised.
2. Identify examples of classification tasks, including the available input features.
3. Apply the Decision Trees algorithm to generate classification rules.

LO's to be achieved:

LON	Lab Outcomes
LO3	Implement the appropriate data mining methods like classification, clustering or association mining on large data sets using open-source tools like WEKA

Theory:

Classification and prediction are two forms of data analysis that can be used to extract models describing important data classes or to predict future data trends. Such analysis can help provide us with a better understanding of the data at large. Whereas *classification* predicts categorical (discrete, unordered) labels, *prediction* models continuous valued functions.

"How does classification work? Data classification is a two-step process:
In the first step, a classifier is built describing a predetermined set of data classes or concepts. This is the learning step (or training phase), where a classification algorithm builds the classifier by analyzing or "learning from" a training set made up of database tuples and their associated class labels. Because the class label of each training tuple *is provided*, this step is also known as supervised learning (i.e., the learning of the classifier is "supervised" in that it is told to which class each training tuple belongs).
This first step of the classification process can also be viewed as the learning of a mapping or function, $y = f(\boldsymbol{X})$, that can predict the associated class label y of a given tuple $\boldsymbol{X}$. In this view, we wish to learn a mapping or function that separates the data classes. Typically, this mapping is represented in the form of classification rules, Decision Trees, or mathematical formulae. In the

second step, we determine if the model's accuracy is acceptable, and if so, we use the model to classify new data.

Classification by Decision Tree Induction

Decision tree induction is the learning of decision trees from class-labeled training tuples. A decision tree is a flowchart-like tree structure, where

- Each internal node (nonleaf node) denotes a test on an attribute,
- Each branch represents an outcome of the test, and
- Each leaf node (or *terminal node*) holds a class label.
- The topmost node in a tree is the root node.

"How are decision trees used for classification?" Given a tuple, ***X***, for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree. A path is traced from the root to a leaf node, which holds the class prediction for that tuple. Decision trees can easily be converted to classification rules.

Why are decision tree classifiers so popular?

- The construction of decision tree classifiers does not require any domain knowledge or parameter setting, and therefore is appropriate for exploratory knowledge discovery. Decision trees can handle high dimensional data.
- Their representation of acquired knowledge in tree form is intuitive and generally easy to assimilate by humans.
- The learning and classification steps of decision tree induction are simple and fast.
- In general, decision tree classifiers have good accuracy.
- Decision trees are the basis of several commercial rule induction systems.
- Decision tree induction algorithms have been used for classification in many application areas, such as medicine, manufacturing and production, financial analysis, astronomy, and molecular biology.

However**, successful use may depend on the data at hand.**

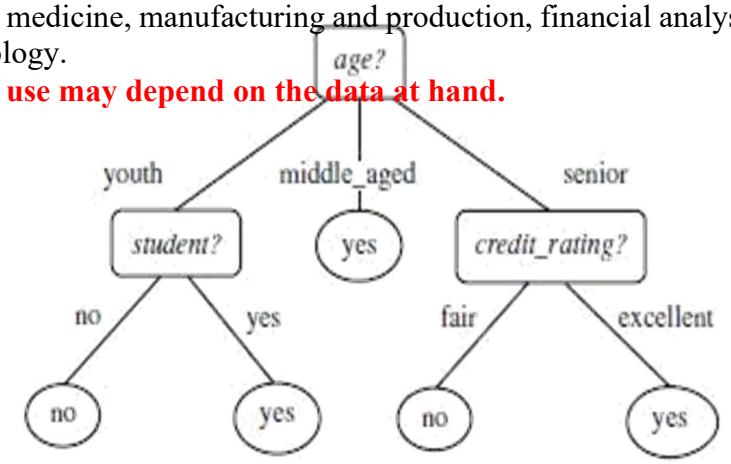


Figure-1: A decision tree for the concept *buys computer*, indicating whether a customer at *AllElectronics* is likely to purchase a computer. Each internal (nonleaf) node represents a test on an attribute. Each leaf node represents a class (either *buys computer = yes* or *buys computer = no*).

Decision Tree Induction

During the late 1970s and early 1980s, J. Ross Quinlan, a researcher in machine learning, developed a decision tree algorithm known as ID3 (Iterative Dichotomiser). Quinlan later presented C4.5 (a successor of ID3), which became a benchmark to which newer supervised learning algorithms are often compared. In 1984, a group of statisticians (L. Breiman, J. Friedman, R. Olshen, and C. Stone) published the book *Classification and Regression Trees* (CART). ID3, C4.5, and CART adopt a greedy (i.e., nonbacktracking) approach in which decision trees are constructed in a top-down recursive divide-and-conquer manner.

Algorithm: Generate decision tree. Generate a decision tree from the training tuples of data partition
D.

Input:

- Data partition, *D*, which is a set of training tuples and their associated class labels;
- *attribute list*, the set of candidate attributes;
- *Attribute selection method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting attribute* and, possibly, either a *split point* or *splitting subset*.

Output: A decision tree.

Method:

(1) create a node *N*;
(2) if tuples in *D* are all of the same class, *C* then
(3) return *N* as a leaf node labeled with the class *C*;
(4) if *attribute list* is empty then
(5) return *N* as a leaf node labeled with the majority class in *D*; // majority voting
(6) Apply Attribute selection method(*D*, *attribute list*) to find the "best" *splitting criterion*;
(7) label node *N* with *splitting criterion*;
(8) if *splitting attribute* is discrete-valued and multiway splits allowed then //
not restricted to binary trees
(9) *attribute list* ← *attribute list - splitting attribute*; // remove *splitting attribute*
(10) for each outcome *j* of *splitting criterion*
// partition the tuples and grow subtrees for each partition
let *Dj* be the set of data tuples in *D* satisfying outcome *j*; // a partition

(12) if *Dj* is empty then
(13) attach a leaf labeled with the majority class in *D* to node *N*;
(14) else attach the node returned by Generate decision tree(*Dj*, *attribute list*) to node
N; endfor
(15) return *N*;

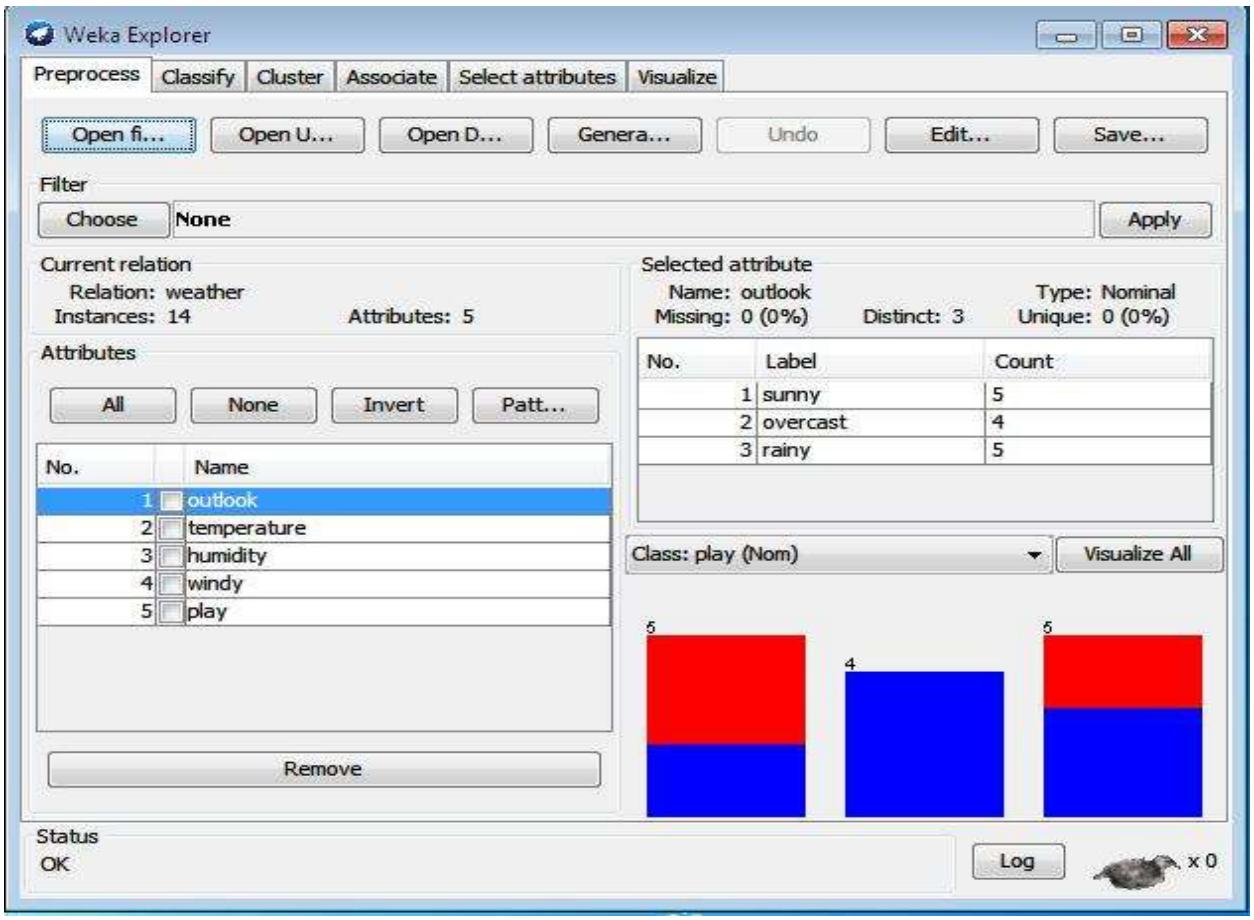
Conclusion:

- The decision tree provides a theoretical framework for taking into account not only the experimental data to design an optimal classifier, but also a structural behavior for allowing better generalization capability.

- Decision trees are also not sensitive to outliers since the splitting happens based on proportion of samples within the split ranges and not on absolute values.

- Highly nonlinear relationships between variables will result in failing checks for simple regression models and thus make such models invalid. However, decision trees do not require any assumptions of linearity in the data. Thus, we can use them in scenarios where we *know* the parameters are nonlinearly related

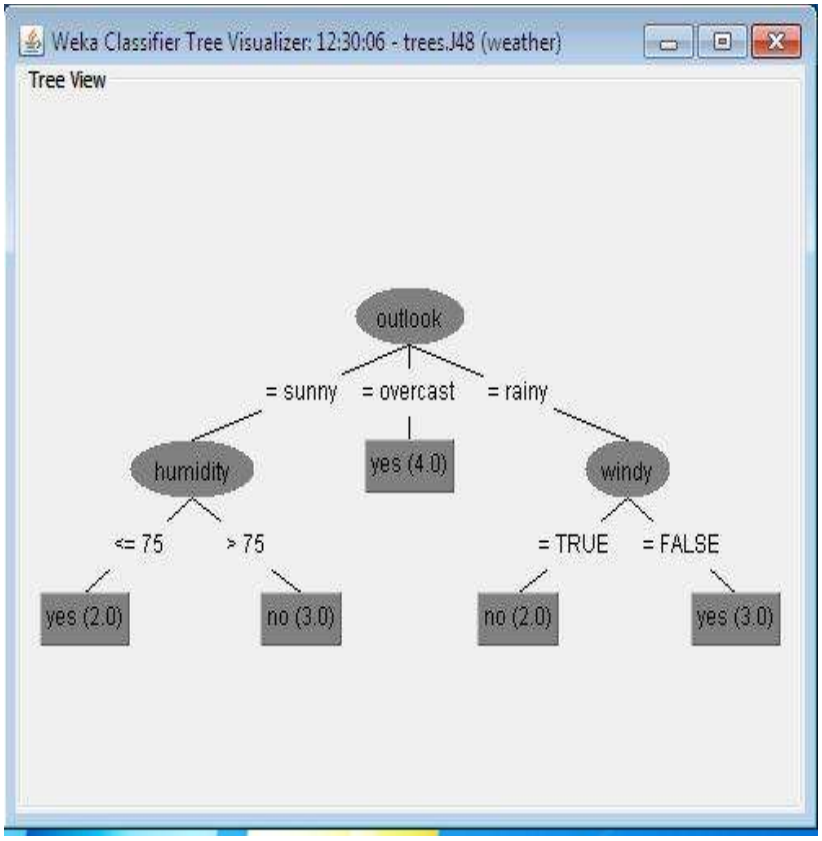
Weka output:



=== Run information ===

Scheme:weka.classifiers.trees.J48 -C 0.25 -M 2
Relation: weather
Instances: 14

Attributes: 5
outlook
temperature
humidity
windy
play



=== Classifier model (full training set) ===

J48 pruned tree
---

outlook = sunny
| humidity <= 75: yes (2.0)
| humidity > 75: no (3.0)
outlook = overcast: yes (4.0)
outlook = rainy
| windy = TRUE: no (2.0)
| windy = FALSE: yes (3.0)

Number of Leaves : 5

Size of the tree : 8

Time taken to build model: 0.02 seconds
=== Summary ===

Correctly Classified Instances	9	64.2857 %
Incorrectly Classified Instances	5	35.7143 %
Mean absolute error	0.2857	
Total Number of Instances	14	

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	Class
	0.778	0.6	0.7	0.778	0.737	0.789	yes
	0.4	0.222	0.5	0.4	0.444	0.789	No
Weighted Avg.	0.643	0.465	0.629	0.643	0.632	0.789	

=== Confusion Matrix ===

```
 a b   <-- classified as
 7 2 | a = yes
3 2 | b = no
```

Post Lab Exercise:

Qu-1: Briefly outline the major steps of *decision tree classification*.
Answer: The major steps are as follows:

- The tree starts as a single root node containing all of the training tuples.
- If the tuples are all from the same class, then the node becomes a leaf, labeled with that class.
- Else, an attribute selection method is called to determine the splitting criterion. Such a method may using a heuristic or statistical measure (e.g., information gain, gain ratio or gini index) to select the "best" way to separate the tuples into individual classes. The splitting
  criterion consists of a splitting attribute and may also indicate either a split-point or a splitting subset, as described below.
- Next, the node is labeled with the splitting criterion, which serves as a test at the node. A branch is grown from the node to each of the outcomes of the splitting criterion and the tuples are partitioned accordingly. There are three possible scenarios for such partitioning. (1) If the splitting attribute is discrete-valued, then a branch is grown for each possible value of the attribute. (2) If the splitting attribute, A, is continuous-valued, then two branches are grown, corresponding to the conditions $A \leq split\ point$ and $A > split\ point$. (3) If the splitting attribute is discrete valued and a binary tree must be produced (e.g., if the gini index was used as a selection measure), then the test at the node is "$A \in SA$?" where SA is the splitting subset for A. It is a subset of the known values of A. If a given tuple has value aj of A and if $aj \in SA$, then the test at the node is satisfied.
- The algorithm recurses to create a decision tree for the tuples at each partition.

- If all tuples at a given node belong to the same class, then transform that node into a leaf, labeled with that class.
- If there are no more attributes left to create more partitions, then majority voting can be used to convert the given node into a leaf, labeled with the most common class among the tuples.

If there are no tuples for a given branch, a leaf is created with the majority class from the parent node.

B) Implementation of Random Forest Classifier using WEKA

Objectives:

After completing this experiment you will be able to:

1. Explain the value of finding associations in market basket data.
2. Characterize the kinds of patterns that can be discovered by association rule mining.
3. Describe how to extend a relational system to find patterns using association rules.

LO's to be achieved:

LON	Lab Outcomes
LO3	Implement the appropriate data mining methods like classification, clustering or association mining on large data sets.

Theory

Recently there has been a lot of interest in "ensemble learning" — methods that generate many classifiers and aggregate their results. Two well-known methods are boosting and bagging of classification trees. In boosting, successive trees give extra weight to points incorrectly predicted by earlier predictors. In the end, a weighted vote is taken for prediction. In bagging, successive trees do not depend on earlier trees — each is independently constructed using a bootstrap sample of the data set. In the end, a simple majority vote is taken for prediction.

Imagine that each of the classifiers in the ensemble is a *decision tree* classifier so that the collection of classifiers is a "forest." The individual decision trees are generated using a random selection of attributes at each node to determine the split. More formally, each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. During classification, each tree votes and the most popular class is returned.

- Random forests can be built using bagging in tandem with random attribute selection. A training set, *D*, of *d* tuples is given. The general procedure to generate *k* decision trees for the ensemble is as follows.
- For each iteration, *i, (i*= 1, 2, : : : , *k*), a training set, *Di* , of *d* tuples is sampled with replacement from *D*. That is, each *Di* is a bootstrap sample of *D*, so that some tuples may occur more than once in *Di*, while others may be excluded.
- Let *F* be the number of attributes to be used to determine the split at each node, where *F* is much smaller than the number of available attributes.
- To construct a decision tree classifier, *Mi*, randomly select, at each node, *F* attributes as candidates for the split at the node.
- The CART methodology is used to grow the trees. The trees are grown to maximum size and are not pruned. Random forests formed this way, with *random input selection*, are called Forest-RI.

Another form of random forest, called Forest-RC, uses *random linear combinations* of the input attributes. Instead of randomly selecting a subset of the attributes, it creates new attributes (or features) that are a linear combination of the existing attributes. That is, an attribute is generated

by specifying L, the number of original attributes to be combined. At a given node, L attributes are randomly selected and added together with coefficients that are uniform random numbers on [-1, 1]. F linear combinations are generated, and a search is made over these for the best split. This form of random forest is useful when there are only a few attributes available, so as to reduce the correlation between individual classifiers.

Breiman proposed random forests, which add an additional layer of randomness to bagging. In addition to constructing each tree using a different bootstrap sample of the data, random forests change how the classification or regression trees are constructed. In standard trees, each node is split using the best split among all variables. In a random forest, each node is split using the best among a subset of predictors randomly chosen at that node. This somewhat counterintuitive strategy turns out to perform very well compared to many other classifiers, including discriminant analysis, support vector machines and neural networks, and is robust against overfitting. Random forests are comparable in accuracy to AdaBoost, yet are more robust to errors and outliers. The generalization error for a forest converges as long as the number of trees in the forest is large. The accuracy of a random forest depends on the strength of the individual classifiers and a measure of the dependence between them. The ideal is to maintain the strength of individual classifiers without increasing their correlation. Random forests are insensitive to the number of attributes selected for consideration at each split. Typically, up to $log_2 d$ +1 is chosen. (An interesting empirical observation was that using a single random input attribute may result in good accuracy that is often higher than when using several attributes.) Because random forests consider many fewer attributes for each split, they are efficient on very large databases. They can be faster than either bagging or boosting. Random forests give internal estimates of variable importance. It is very user-friendly in the sense that it has only two parameters (the number of variables in the random subset at each node and the number of trees in the forest), and is usually not very sensitive to their values.

<u>The advantages of random forest are:</u>

- It is one of the most accurate learning algorithms available. For many data sets, it produces a highly accurate classifier.
- It runs efficiently on large databases.
- It can handle thousands of input variables without variable deletion.
- It gives estimates of what variables are important in the classification.
- It generates an internal unbiased estimate of the generalization error as the forest building progresses.
- It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.
- It has methods for balancing error in class population unbalanced data sets.
- Generated forests can be saved for future use on other data.
- Prototypes are computed that give information about the relation between the variables and the classification.
- It computes proximities between pairs of cases that can be used in clustering, locating outliers, or (by scaling) give interesting views of the data.

- The capabilities of the above can be extended to unlabeled data, leading to unsupervised clustering, data views and outlier detection.
- It offers an experimental method for detecting variable interactions.

Disadvantages

- Random forests have been observed to overfit for some datasets with noisy classification/regression tasks.
- For data including categorical variables with different number of levels, random forests are biased in favor of those attributes with more levels. Therefore, the variable importance scores from random forest are not reliable for this type of data.

Conclusion

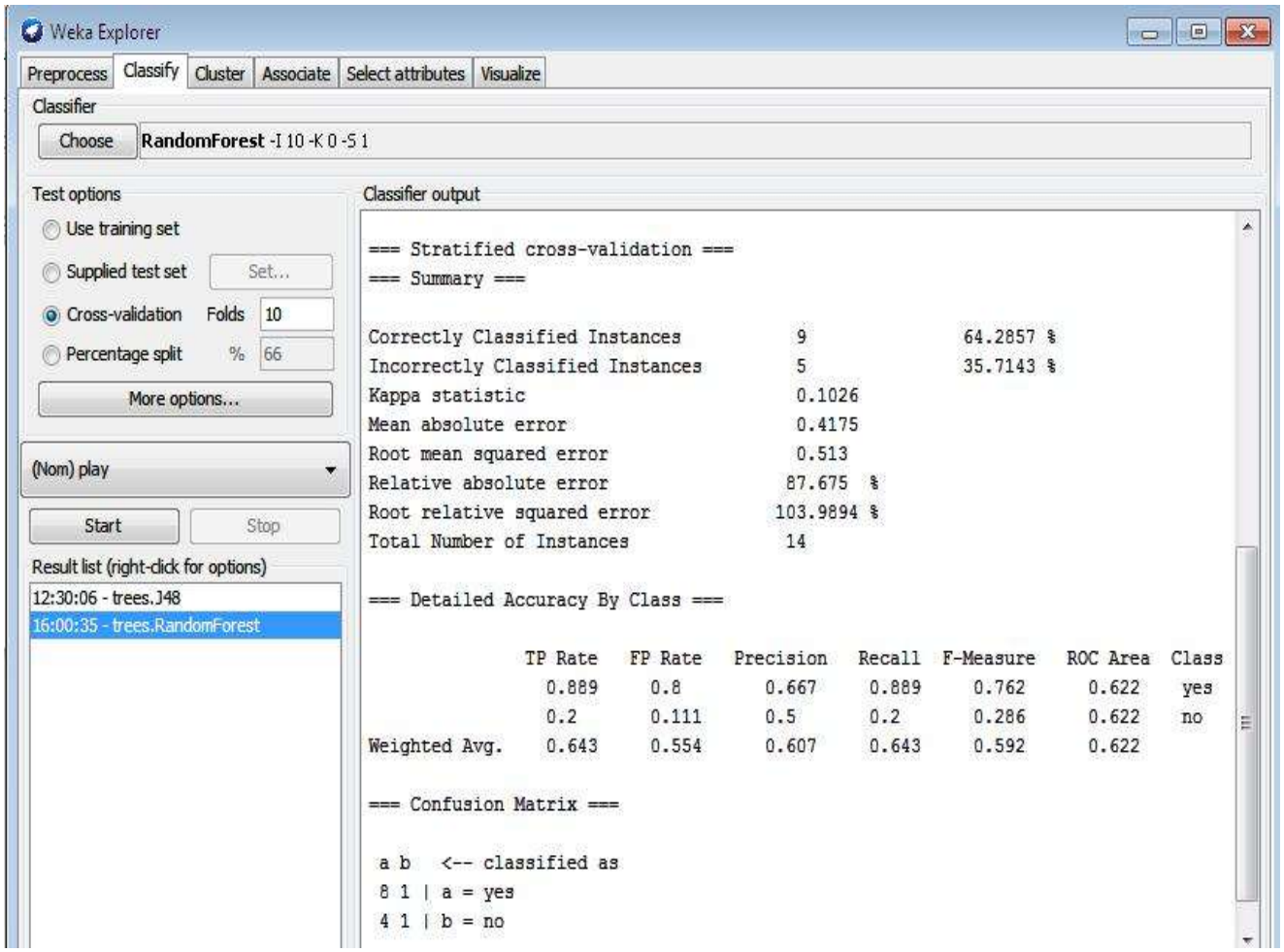
- Fast processing and results!
  - RF is fast to build. Even faster to predict!
  - Practically speaking, not requiring cross-validation alone for model selection significantly speeds training by 10x-100x or more.
  - Fully parallelizable … to go even faster!
- Automatic predictor selection from large number of candidates
- Resistance to over training
- Ability to handle data without preprocessing
  - data does not need to be rescaled, transformed, or modified
  - resistant to outliers
  - automatic handling of missing values

Cluster identification can be used to generate tree-based clusters through sample proximity.

Weka Output:

```
=== Run information ===
Scheme:weka.classifiers.trees.RandomForest -I 10 -K 0 -S 1
Relation:     weather
Instances:    14
Attributes:   5
              outlook
              temperature
              humidity
              windy
              play
Test mode:10-fold cross-validation
=== Classifier model (full training set) ===
Random forest of 10 trees, each constructed while considering 3 random features.
Out of bag error: 0.4286
```

Time taken to build model: 0.01 seconds



Post Lab Exercise:

Qu-1: Outline methods for addressing the *class imbalance problem*. Suppose a bank would like to develop a classifier that guards against fraudulent credit card transactions. Illustrate how you can induce a quality classifier based on a large set of non-fraudulent examples and a very small set of fraudulent cases.

Answer:

There are in total four general approaches for improving the classification accuracy of class-imbalanced data like the fraudulent credit card transactions data. The first one is **oversampling**, which works by resampling the positive tuples so that the resulting training set contains an equal number of positive and negative tuples. So for the data in the question, we can replicate tuples of the fraudulent cases to form a training set with size comparable to non fraudulent examples.

The second one is **under sampling**, which works by decreasing the number of negative tuples. It randomly eliminates tuples from the majority (negative) class until there are an equal number of positive and negative tuples. Thus for the fraudulent credit card transactions problem, we need to randomly remove the non-fraudulent examples until the size is equal to the fraudulent cases.

The third one is threshold-moving, which applies to classifiers that, given an input tuple, return a continuous output value. Rather than manipulating the training tuples, this method returns classification decision based on the output values. By tuning the threshold of the output value deciding the exact predictions of the input, rare class tuples like the fraudulent cases in the question are easier to classify. And hence, there is less chance of costly false negative errors. In other words, the classifier will hardly misclassify the fraudulent cases.

The final one is Ensemble methods, applied to algorithms like boosting and random forest. The individual classifiers within these algorithms may include versions of the approaches described above, such as oversampling and threshold-moving.